Topic 1. Classes and Objects. Static. Final. Enum. BigDecimal. LocalDate. Abstract Classes. Interfaces.

Class: RecordingStudio

The RecordingStudio class represents a studio that can be rented for a specified number of hours per day. It includes the following **data members**:

1. **Identification Number (id)**
   - A unique identifier for the studio.
   - **Immutable** after the studio object is created.
2. **Hourly Rental Price (priceInBGN)**
   - The cost of renting the studio for **one hour** (in Bulgarian Lev).
   - Must be **greater than or equal to** the minimum rental price.
3. **Maximum Daily Rental Hours (maxHoursPerDay)**
   - Specifies the **maximum** number of hours the studio can be rented per day.
   - Must be between **0 and 24**.
4. **Rented Hours (rentedHours)**
   - The number of hours the studio is rented for on a given day.
   - Cannot exceed maxHoursPerDay.
5. **Minimum Rental Price (minPriceInBGN)**
   - A **constant value** applicable to all studios.
   - Defines the lowest allowed price for renting the studio per hour.
6. **Exchange Rate (exchangeRateEURtoBGN)**
   - The **conversion rate** from Bulgarian Lev (BGN) to Euro (EUR).
   - Used to calculate rental income in **Euros**.

Functionalities to Implement

1. **Set Rental Price (setPrice)**
   - Updates the hourly rental price.
   - If the provided price is **less than the minimum price**, it is automatically set to minPriceInBGN.
2. **Set Rented Hours (setRentedHours)**
   - Sets the number of hours the studio is rented for that day.
   - The value must be **≤ maxHoursPerDay**.
   - If an invalid number is provided, an **exception** should be thrown.
3. **Calculate Income in BGN (getIncomeInBGN)**

    - Returns the **total income in BGN** based on the rented hours and hourly price.
4. **Calculate Income in EUR (`getIncomeInEUR`)**
    - Converts the income to **Euros** using `exchangeRateEURtoBGN`.
5. **Compare Two Studios (`compareTo`)**
    - Compares two `RecordingStudio` objects based on their **income**.

Requirements

- **Exception Handling**:
    - Implement **error handling** where necessary (e.g., negative values, exceeding max hours).
- **Object-Oriented Principles**:
    - Use **abstract classes and interfaces** to promote **code reusability** and **flexibility**.